

Summary of “Towards End-to-End Optimization of LLM-based Applications with Ayo”

Shivam Arora (arshiv), Aman Soni (amansoni), Leonard Zhang (zle), Dimash Umirbayev (dimashu)

Problem and Motivation

Large Language Model (LLM)-based applications consist of both LLM and non-LLM components, such as retrieval, embedding, indexing, and external API calls. While prior work has focused heavily on optimizing LLM inference latency, the overall **end-to-end latency** of these applications remains high. Empirical observations show that non-LLM components can contribute a significant portion of total latency, in some cases exceeding 50%.

Existing orchestration frameworks such as LangChain and LlamaIndex adopt a coarse-grained design, where applications are structured as chains of independent modules. These systems execute modules sequentially and perform scheduling at the request level. Moreover often they do not consider dependencies across the workflow. As a result, they fail to exploit opportunities for parallelism and pipelining, leading to suboptimal resource utilization and higher latency.

The key motivation of this work is to enable **end-to-end optimization** of LLM-based applications by **moving beyond module-level abstractions and capturing fine-grained dependencies within workflows**.

Related Works

Orchestration Frameworks: Frameworks such as LangChain and LlamaIndex simplify application development by chaining steps into modules like retrieval, LLM inference, and post-processing. However, they operate at a coarse granularity and treat each module as a black box. This limits optimization to within individual modules and prevents cross-module optimization.

LLM Inference Optimization: Prior work has proposed various techniques to optimize LLM inference, including batching, caching, and decoding optimizations.

Systems such as vLLM exploit techniques like continuous batching and optimized memory management (KV caching) to reduce redundant computation during decoding. While effective, these approaches focus only on the LLM component and do not address inefficiencies arising from interactions between different components in the workflow.

Scheduling Systems: Existing systems typically perform request-level scheduling, optimizing latency for individual requests. Triton-style systems batch requests based on arrival order and

use fixed batching strategies. They do not consider dependencies between tasks. As a result, these systems fail to exploit workflow-level information. This leads to suboptimal batching and scheduling decisions from an application-level perspective.

Solution Overview

Ayo proposes a fine-grained orchestration framework that models LLM-based applications as primitive-level dataflow graphs to enable end-to-end optimization. Ayo operates in two stages: (1) an offline stage where developers define execution engines and workflow templates, and (2) an online stage where each query is converted into a primitive-level graph, optimized, and executed.

Primitive-Level Dataflow Graph

Instead of treating modules as atomic units, Ayo decomposes workflows into smaller units called task primitives, such as embedding, searching, prefilling, decoding, and reranking. Each query is represented as a primitive-based dataflow graph (p-graph), where nodes correspond to primitives and edges represent data dependencies.

This representation exposes fine-grained dependencies within the workflow, enabling opportunities for parallelism and pipelining across components that are not visible in module-based abstractions.

Type	Description
Reranking	Compute and rank the relevance scores for the query and context pairs.
Ingestion	Store embedding vectors into vector database
Searching	Perform vector searching in the database
Embedding	Create embedding vectors for docs or questions
Prefilling	The prefilling part of LLM inference
Decoding	The decoding part of LLM inference
Partial Prefilling	Prefilling for partial prefix of a prompt (e.g. instruction, context, question)
Full Prefilling	Prefilling for rest part of a prompt after a partial prefilling
Partial Decoding	Part of full decoding for partial output
Condition	Decide the conditional branch
Aggregate	Aggregate the results from multiple primitives

Figure 1: Examples of some primitives. White backgrounds denote common operations, blue for decomposed operations, and gray for control flow operations.

Graph Optimization

Ayo transforms the primitive-level graph into an optimized execution graph (e-graph) using several optimization passes:

- **Dependency Pruning:** Removes unnecessary dependencies to enable parallel execution of independent primitives.
- **Stage Decomposition:** Splits large batchable tasks into smaller stages, allowing pipeline parallelism across primitives.
- **Prefilling Split:** Separates LLM prefilling into partial and full stages, enabling early computation before all inputs are available. It is done by overlapping static instruction prefilling with upstream I/O tasks.
- **Decoding Pipelining:** Streams partial decoding outputs to downstream tasks, allowing overlap between decoding and subsequent operations.

These optimizations together increase parallelism, reduce idle time, and improve end-to-end execution efficiency.

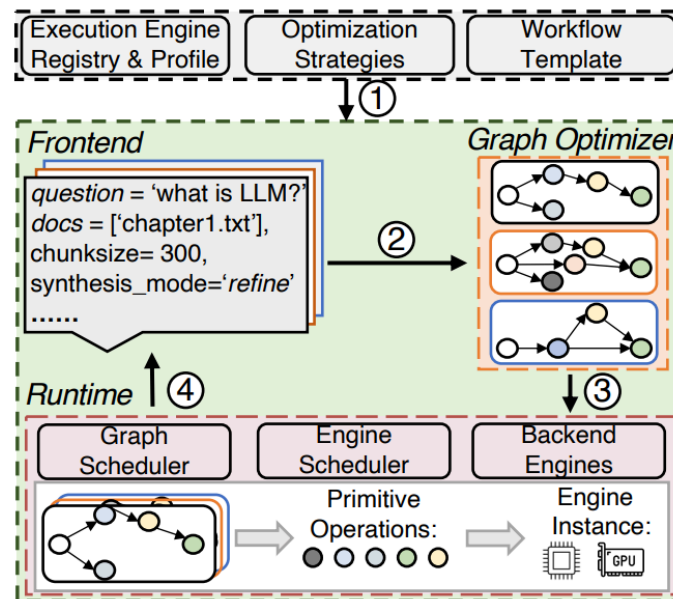


Figure 2: System overview of Ayo

Runtime Scheduling

Ayo employs a two-tier scheduling architecture consisting of a graph scheduler and an engine scheduler. The graph scheduler tracks dependencies in the execution graph and dispatches primitives once their dependencies are satisfied. The engine scheduler, on the other hand,

manages execution engines and is responsible for batching requests and allocating resources efficiently across different primitives.

A key feature is **application-aware scheduling**. This leverages primitive dependencies and graph topology to make more efficient batching decisions compared to naive request-level scheduling.

Overall, Ayo achieves up to **2.09x** improvement in end-to-end latency compared to existing systems.

Limitations

- The system relies on predefined workflows and performs optimizations ahead of schedule. It lacks a just in time approach and hence would not perform well with dynamic workflows. It wouldn't be able to capture the complete low level dataflow graph before execution.
- Ayo requires modifications to execution engines and developer-defined workflow templates, introducing additional engineering overhead. There is a trade-off between finer grained optimizations and developer overhead.
- The system does not explicitly identify or prioritize critical path operations, which could further improve scheduling efficiency. There is room for improvement if less critical nodes are computed before higher priority ones.

Future Research Directions

The paper suggests several directions for future research:

1. **Support for Dynamic Workflows:** Extending Ayo to handle dynamically generated workflows, such as those in agent-based systems.
2. **Critical Path Analysis:** Incorporating critical path-aware scheduling to prioritize latency-sensitive operations.
3. **Improved Scheduling Strategies:** Developing more advanced scheduling policies for better resource allocation and load balancing.

Summary of “AVA: Towards Agentic Video Analytics with Vision Language Models”

Shivam Arora (arshiv), Aman Soni (amansoni), Leonard Zhang (zle), Dimash Umirbayev (dimashu)

Problem and Motivation

Video analytics can be useful for many important detection and monitoring tasks like environmental monitoring, surveillance, and automation such as self-driving cars.. Vision Language Models (VLMs) that can perform these tasks are severely limited by their context window sizes. Due to how large video sizes are, even a few hundred or few thousand frames can quickly fill up the context window, limiting the usefulness of current VLMs in these tasks. For instance, the VLM QwenVL can only fit 768 frames, making it impossible to use for long-term video analytics.

The paper describes 5 levels of intelligence when it comes to video analytics systems. Simpler levels like L1-L3 focus on closed-ended and pre-defined tasks like identifying pedestrians, or responding to queries like “What animals appeared in this video.” L4 and L5 are much more sophisticated and enable the use of open-ended queries, with a significant improvement in comprehension and reasoning. An L4 system would be able to answer questions like “Are any animals exhibiting unusual behavior?” making them vastly more useful for these tasks.

Because of both limited context window sizes allowing only short videos to be analyzed, as well as the need for more advanced querying methods to interact with that data, this motivates the need for a system capable of handling long videos as well as addressing open-ended and complex queries.

Related Works

Video Language Models (VLM): Video Language Models are models that are capable of processing videos. They tokenize each frame given, in order to construct an understanding of causality and comprehend events in the video. However VLMs are severely limited by context window size, limiting their usefulness to videos that are at most a few minutes long. Typically, not every frame is needed in order to understand a video’s contents, so methods to remove unnecessary information (i.e. reducing resolution, reducing frame rate) can help with this issue and increase the usable video length. However, the authors of this paper did not find this to be enough to substantially increase the performance and understanding of a VLM, requiring other methods.

Retrieval Augmented Generation (RAG): Following from the key insight that specific queries can be answered without needing to see every frame, retrieval augmented generation is a method where only specific, relevant frames are retrieved. These frames, along with the query, are inputted into a VLM in order to respond to the query. Each frame is tagged with a vector embedding, containing information that describes the contents of that frame. Frames are then retrieved by comparing the query embedding to the frame embeddings, and retrieving the ones

that are similar. This method has the drawback that the quality of the response depends on the quality of the embedding used to retrieve the frames as well as poor understanding of overall context, leading to poor performance with multi-hop queries or more open-ended queries.

Solution Overview

For real-time video processing, AVA indexes the data by using an event knowledge graph (EKG) with semantic chunking. Event knowledge graphs can accurately capture key events within the video and form meaningful relationships and interactions between events. Semantic chunking is used to process the video. The video is first divided into small, equal-sized sections that are each 3 seconds. These sections are fed into a VLM to generate descriptions of the events in each section. Sections are then combined if they are similar, which is determined by feeding them into a text embedding model and comparing to nearby sections. This creates sections that are semantically meaningful and ties specific events to specific frames.

After the video is processed, it can be retrieved and used to answer queries. AVA uses a tri-view retrieval method where 3 types of information are provided: knowledge of events, knowledge of entities, and the raw video embedding. This information is retrieved by comparing query embeddings to section embeddings to determine which chunks are relevant. AVA uses an agentic search that mimics human search behavior by creating 3 search actions: forward (look at subsequent events), backward (look at previous events), and re-query (search for related or similar events). The initial query returns a set of nodes in the EKG that begins the tree search. At each searched node, the 3 search actions are performed as well as a summary-and-answer action which answers the query using a chain-of-thought prompting method, based on the knowledge known to it. This search continues until the max depth of 3 is reached. This iterative search thoroughly searches the EKG for relevant information and constructs a multitude of answers.

From these answers, the process to determine which answer is best is done by ranking them on quality and consistency. Answers that appear multiple times are scored highly in terms of answer agreement, showing consensus, while the use of chain of thought prompting helps ensure that the logical reasoning behind the answer is solid. AVA selects the top 2 differing answers and feeds them both back into a VLM, along with the video frames corresponding to the events to generate a final answer.

AVA was evaluated on multiple datasets and consistently outperformed current state of the art video analytics systems. It achieved an improvement of 16.9% on LVBench, 5.2% on VideoMME-Long, and 26.9% on AVA-100 compared to existing solutions.

Limitations

- **Slow processing speed:** While AVA aims to be real-time, results showed that it was able to process about 2.5 FPS on an RTX 3090 GPU. For many applications, 2.5 FPS is simply not enough, and increasing the frame rate would increase latency, making the system not real-time.
- **Latency:** For processing a query, the latency was found to be up to around 3 minutes, with the majority of the time being taken up by agentic searching. This limits the use case of AVA and prevents it from being used in certain real-world applications.

Future Research Directions

- **Performance:** Improving processing speed and latency could be an important next step to enable video analytics technology to be used in many more real-world applications.
- **Correctness:** While AVA set many records, the overall accuracy was still relatively low on many datasets (~60% accuracy) which is likely too low for use in many real-world applications. Efforts to improve accuracy could be very important for video analytics technology.

Summary of Class Discussion

Q1: AVA: How big can the EKG be for a really long video? Does it need additional memory management if it gets really big?

Ans: It's not that big because of semantic chunking which removes a lot of redundancy.

Q2: AVA can get really high throughput because it uses a small model for indexing, but are there any consequences of using a small model?

Ans: No, because the answer is checked later on by a bigger, more powerful model. The small model is used to generate quick descriptions but this task doesn't require large models.

Q3: AVA: Throughout the construction of the model, they consciously chose when to use big or small models but latency is still really high. Did they compare AVA's latency to other models?

Ans: They did not provide comparisons with other models in terms of latency.

Q4: AVA: If latency is really slow already, why not use a bigger model to possibly get more accuracy?

Ans: it would probably be even worse.

Q5: Ayo: A smaller batch can be scheduled sooner, more efficient scheduling but smaller batches get less GPU utilization. Does Ayo explicitly address the trade-off between smaller and larger batch sizes in scheduling ?

Ans: Ayo implicitly captures the trade-off between smaller and larger batch sizes through its application-aware scheduling design, but it does not explicitly quantify this trade-off. Smaller batches can be scheduled earlier and reduce waiting time. This leads to lower latency for

individual primitives. In contrast, larger batches improve GPU utilization but may introduce additional delay if the scheduler waits to accumulate enough requests to fill the batch. For example, if batch size=8 but the incoming request has batch size=6, this will incur additional latency since the process has to wait for the two additional batches to process. Ayo mitigates this by leveraging workflow-level dependency information to make more informed batching decisions. This is the key element of the paper.

Q6: Ayo: Does Ayo operate on single queries or multiple queries, and why does this distinction matter?

Ans: Ayo is designed to operate in a multi query setting. Multiple queries are processed concurrently on a shared cluster. Queries arrive online and are independently transformed into e graphs before scheduling. This setting enables batching and resourcing sharing across queries. Additionally, Ayo can make application aware scheduling by considering dependencies which improves overall performance. The approach prioritizes end to end latency of the workflow rather than per request efficiency.

Q7: Ayo: One of the key limitations is that Ayo can't handle dynamic workflows due to having to know the graph ahead of time, could this be improved to work on dynamic workflows?

Ans: This is one of the main limitations of Ayo. Ayo assumes that workflows are defined ahead of time, requiring developers to specify execution engines, workflow templates, and configuration details prior to execution. Supporting dynamic workflows would require the system to constrict and optimize graphs on the fly. This introduces additional complexity and engineering overhead. Model driven probabilistic approaches might help but that remains an open challenge and area for future work.

Q8: AVA: Accuracy can be different based on the type of question asked (summarize the video vs. what happened at a certain time). Would runtime (throughput/latency) be different as well?

Ans: It would still take a similar amount of time because most of the time is consumed by agentic search.

Q9: AVA: Does the type of video affect performance? A video of a busy street compared to a video of a wall would result in a large difference in the number of key frames. Would this lead to longer agentic search times for the video with lots of content?

Ans: The paper doesn't distinguish between the type of video in its evaluation on performance.

Q10: AVA: What is borda counting?

Ans: It is a fancy name for counting and grouping the semantically similar frames in the semantic chunking section of AVA.

Q11: AVA: One of the main contributions of AVA was defining the L1-L5 video intelligence levels. AVA is L4, but what separates it from L5?

Ans: An L5 would also need access to open-source knowledge like web access, not just the video contents. This would be some kind of open-ended knowledge graph. For example, adding MCP to AVA can potentially make it into an L5 video intelligence.